Lab 4: Querying Audit Logs in Log Analytics

Description

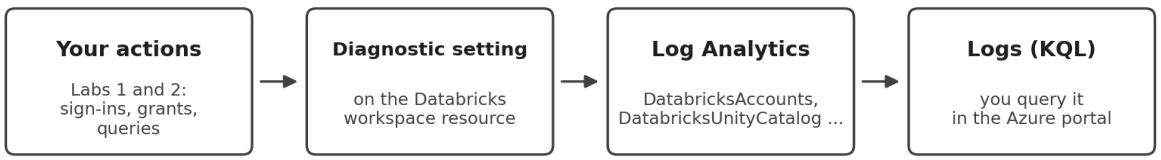
Your trainer has set up a diagnostic setting that sends the workspace's audit events to a Log Analytics workspace. In this lab you find your Lab 1 and Lab 2 activity there with KQL, and compare what Log Analytics receives with the system table from Lab 3.

Learning Objectives

- Explain how diagnostic settings deliver Databricks audit logs to Azure
- Find the Databricks log tables in Log Analytics
- Query sign-ins, permission changes and denied requests with KQL
- Identify which events reach Log Analytics and which do not

Step by Step Guide

How it works



Workspace-level events only. Arrives within minutes. Retention is set by your team.

Your lab values

Item	Value
Prefix	Same as Labs 1 to 3
Your learner email	For example bob@saurabhuptut.onmicrosoft.com
Your catalog	<prefix>_catalog
Log Analytics workspace	Name provided by your trainer

Tip: Replace <prefix> and <your-learner-email> before running. Run one query at a time: place the cursor inside it and click Run.

Step 1: See where the logs are sent

1. Sign in to the Azure portal with your learner account.
2. Open the Databricks workspace resource > **Monitoring** > **Diagnostic settings**. Open the setting and look at it. Do not change it.

> **Check:** The setting sends log categories such as accounts and unityCatalog to the Log Analytics workspace. If you cannot open this page, your trainer will show it.

Step 2: Open Log Analytics

1. Search the portal for **Log Analytics workspaces** and open the workspace named by your trainer.
2. Click **Logs**. Close the Queries window if it opens and switch to **KQL mode**.
3. Set the time range to **Set in query** or **Last 30 days**.

Step 3: Find the Databricks tables

```
union withsource = TableName Databricks*
| where TimeGenerated > ago(30d)
| summarize Events = count() by TableName
| order by Events desc
```

> **Check:** You see tables such as DatabricksAccounts, DatabricksUnityCatalog and DatabricksDatabricksSQL. Each log category has its own table.

Step 4: Summarise everything you did

```
union Databricks*
| where TimeGenerated > ago(30d)
| extend Email = tostring(parse_json(Identity).email)
| where Email == "<your-learner-email>"
| summarize Events = count() by ServiceName, ActionName
| order by Events desc
```

> **Check:** Compare this with Step 3 of Lab 3. Most actions match, but the list is shorter.

Step 5: Who signed in as your test users? (Lab 1)

```
DatabricksAccounts
| where TimeGenerated > ago(30d)
| where ActionName in ("aadBrowserLogin", "login", "aadTokenLogin", "tokenLogin")
| extend Email = coalesce(tostring(parse_json(Identity).email),
                          tostring(parse_json(RequestParams).user))
| where Email startswith "<prefix>-"
| project TimeGenerated, Email, ActionName, SourceIPAddress,
          Status = tostring(parse_json(Response).statusCode)
| order by TimeGenerated desc
```

> Check: The same sign-ins you found in Step 4 of Lab 3.

Step 6: Who changed permissions on your catalog? (Lab 2)

```
DatabricksUnityCatalog
| where TimeGenerated > ago(30d)
| where ActionName == "updatePermissions"
| where RequestParams has "<prefix>_catalog"
| extend Params = parse_json(RequestParams)
| project TimeGenerated,
          Who = tostring(parse_json(Identity).email),
          Object = tostring(Params.securable_full_name),
          Changes = tostring(Params.changes)
| order by TimeGenerated asc
```

> Check: Each GRANT and REVOKE from Lab 2 appears.

Step 7: Find denied requests on your catalog

```
DatabricksUnityCatalog
| where TimeGenerated > ago(30d)
| where RequestParams has "<prefix>_catalog"
| extend Status = toint(parse_json(Response).statusCode)
| where Status != 200
| project TimeGenerated,
          Who = tostring(parse_json(Identity).email),
          ActionName, Status,
          Error = tostring(parse_json(Response).errorMessage)
| order by TimeGenerated desc
```

> Check: analyst1's failed attempts from Lab 2 appear, as in Step 8 of Lab 3.

Step 8: Compare with the account console actions

In Step 5 of Lab 3 you listed your account console actions from Lab 1 (workspace_id 0). Now list your account and group actions here:

```
DatabricksAccounts
| where TimeGenerated > ago(30d)
| where tostring(parse_json(Identity).email) == "<your-learner-email>"
| summarize Events = count() by ActionName
| order by Events desc
```

> Check: The account console events from Lab 3, Step 5 are not delivered here, because diagnostic settings only send workspace-level events. Any similar action names you see come from things you did inside the workspace, such as setting entitlements.

Step 9 (Optional): Chart your activity

```
union Databricks*
| where TimeGenerated > ago(30d)
| where tostring(parse_json(Identity).email) startswith "<prefix>"
| summarize Events = count() by bin(TimeGenerated, 1h), Type
| render timechart
```

> Check: A chart shows when you and your test users were active during Labs 1 and 2.

Outcomes

- You found the Databricks audit tables in Log Analytics
- You traced sign-ins, permission changes and denied requests with KQL
- You know that Log Analytics receives workspace-level events only
- You can explain when to use the system table and when to use Log Analytics